XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

EEE312 Wee6

Lab Practice

Write a C program to solve a system of 4 linear equations using the Jacobi method. Show the progress of the iterative steps by displaying the values of the variables **x** at every iteration.

.

```
// Beginning of the C Source code
```

```c
// Jacobi method to solve linear equations

#include <stdio.h>
#include <math.h>
#define N 4

void Jacobi(double A[][N], double x[N], double b[N]);
void printMatrix(double A[][N]);
void printVector(double b[N]);
```

Function Jacobi()

```
28  void Jacobi(double A[][N], double x[N], double b[N])
29  {
30      double x_new[N] = {0.0}; // new updated x
31      double error = 0.0;
32      double epsilon = 0.005;
33
34      do{ // beginning of the iterative calculation
35          float sum = 0.0;
36          for(int i=0; i<N; i++)
37          {
38              sum = 0.0;
39              for(int j = 0; j<N; j++)
40              {
41                  if(i != j) sum += A[i][j]*x[j];// calculate the sum
42              }
43              x_new[i] = (b[i]-sum)/A[i][i];// update new value of x[i]
44          }
```

Line 30-32: Declare variables x_new[N], error and epsilon.

Line 34: Beginning of the iterative calculation using the do-whileloop.

Line 36-43: Implement the update equation for x values.

The update equations are:

$$x_0^{(k+1)} = \frac{b_0 - a_{01}x_1^{(k)} - a_{02}x_2^{(k)} - a_{03}x_3^{(k)}}{a_{00}}$$

$$x_1^{(k+1)} = \frac{b_1 - a_{10}x_0^{(k)} - a_{12}x_2^{(k)} - a_{13}x_3^{(k)}}{a_{11}}$$

$$x_2^{(k+1)} = \frac{b_2 - a_{20}x_0^{(k)} - a_{21}x_1^{(k)} - a_{23}x_3^{(k)}}{a_{22}}$$

$$x_3^{(k+1)} = \frac{b_3 - a_{30}x_0^{(k)} - a_{31}x_1^{(k)} - a_{32}x_2^{(k)}}{a_{33}}$$

C code Implementation:

x_new[0] = (b[0]-A[0][1]*x[1]-A[0][2]*x[2]-A[0][3]*x[3])/A[0][0];

x_new[1] = (b[1]-A[1][0]*x[0]-A[1][2]*x[2]-A[1][3]*x[3])/A[1][1];

x_new[2] = (b[2]-A[2][0]*x[0]-A[2][1]*x[1]-A[2][3]*x[3])/A[2][2];

x_new[3] = (b[3]-A[3][0]*x[0]-A[3][1]*x[1]-A[3][2]*x[2])/A[3][3];

Alternatively, the update equation is:

$$x_i^{(k+1)} = \frac{b_i - \sum_{j=0, j\neq i}^{N-1} a_{ij} x_j^{(k)}}{a_{ii}}$$

for i= 0, 1, 2, and 3.

C code implementation:

```
36          for(int i=0; i<N; i++)
37          {
38              sum = 0.0;
39              for(int j = 0; j<N; j++)
40              {
41                  if(i != j) sum += A[i][j]*x[j];// calculate the sum
42              }
43              x_new[i] = (b[i]-sum)/A[i][i];// update new value of x[i]
44          }
```

```
45          for(int i=0; i<N; i++)
46          {
47              printf("%f\t", x_new[i]);// print the newly updated value x[i]
48          }
49
50          error = 0.0;
51          for(int i=0; i<N; i++)
52          {
53              if(fabs(x_new[i]-x[i])>error) // find the greatest difference between
54                  error = fabs(x_new[i]-x[i]); // current and updated value
55          }
56          printf("\nError = %f\n", error);
```

Line 45-47: Print out the newly updated values of x_new[i].

Line 51-54: Find the maximum of the difference between xi(k+1)and xi(k)according to the equation:

$$\text{error} = \max(|x_i^{(k+1)} - x_i^{(k)}|) \text{ for } i = 0, 1, 2 \text{ and } 3$$

```
57
58          for(int i = 0; i<N; i++)
59          {
60              x[i] = x_new[i]; // store updated value for next iteration
61          }
62      }while(error>epsilon); // repeat if error > epsilon
63  }
```

Line 58-61: Store the newly updated values x_new[i]in x[i]for the next iteration of the algorithm.

Line 62: Check if the error > epsilon. If yes, continue from the beginning of the do-whileloop. If no, escape out from the do-whileloop.

Function main()

```
11  int main()
12  {
13      double A[N][N] = {{6,-1,2,2},{1,-5,1,-1},{-2,1,-5,1},{4,-2,1,-8}}; //  coefficient matrix A
14      double b[N] = {-1, 3, -2, 2}; //  matrix b
15      double x[N] = {0.0}; // unknown variables
16
17      printf("The original matrix A is \n" );
18      printMatrix(A); // print the original matrix A
19      printf("The original matrix b is \n" );
20      printVector(b); // print the original matrix b
21      printf("\n");
22
23      Jacobi(A, x, b); // Jacobi method
24
25      return 0;
26  }
```

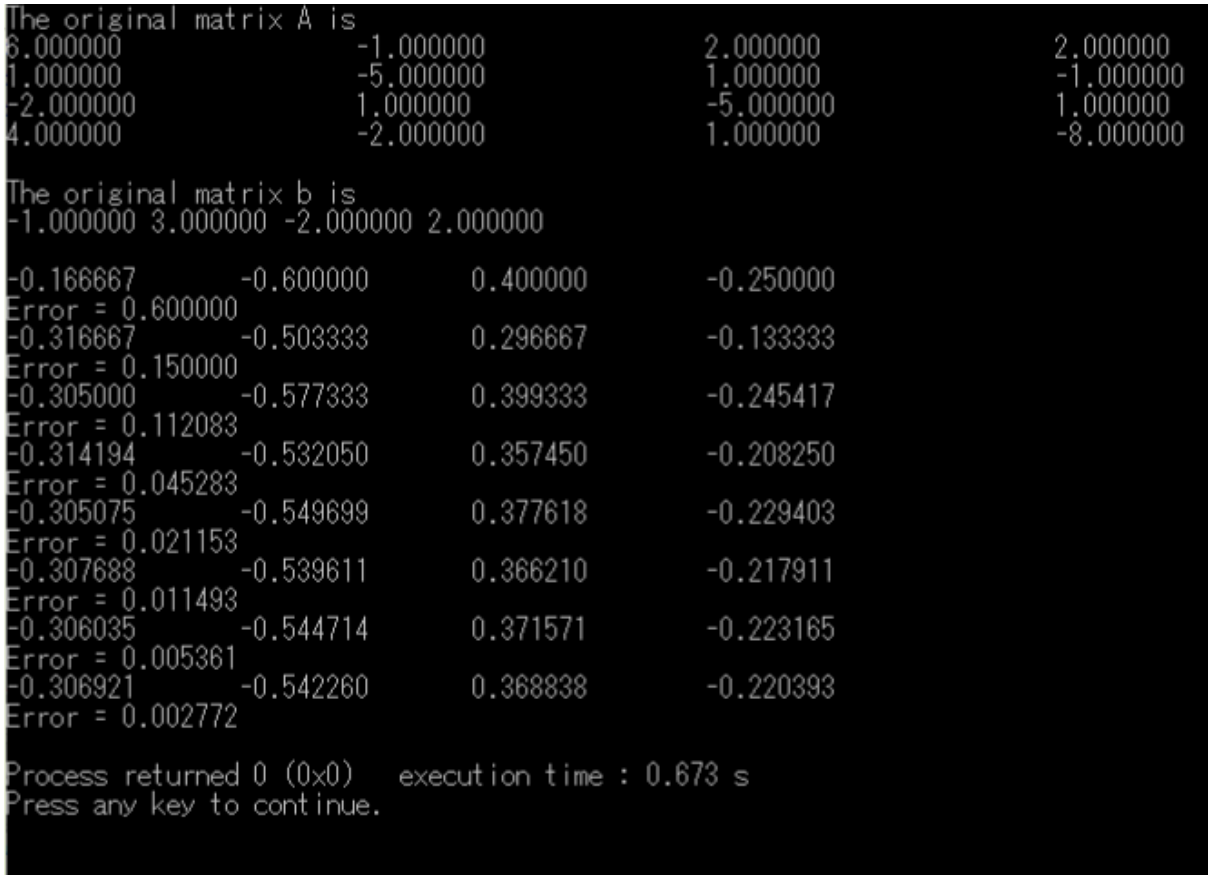
Line 13-15: Declare and initialize A, x and b.

Line 18: Print out matrix A at the program output.

Line 20: Print out vector bat the program output.

Line 23: Carry out the Jacobi method calculations.

Output


The original matrix A is
6.000000 -1.000000 2.000000 2.000000
1.000000 -5.000000 1.000000 -1.000000
-2.000000 1.000000 -5.000000 1.000000
4.000000 -2.000000 1.000000 -8.000000
The original matrix b is
-1.000000 3.000000 -2.000000 2.000000
-0.166667 -0.600000 0.400000 -0.250000
Error = 0.600000
-0.316667 -0.503333 0.296667 -0.133333
Error = 0.150000
-0.305000 -0.577333 0.399333 -0.245417
Error = 0.112083
-0.314194 -0.532050 0.357450 -0.208250
Error = 0.045283
-0.305075 -0.549699 0.377618 -0.229403
Error = 0.021153
-0.307688 -0.539611 0.366210 -0.217911
Error = 0.011493
-0.306035 -0.544714 0.371571 -0.223165
Error = 0.005361
-0.306921 -0.542260 0.368838 -0.220393
Error = 0.002772
Process returned 0 (0x0) execution time : 0.673 s
Press any key to continue.